

# Reviewer Pack — Minimal Rank of Tropicalized Automorphic Representations vs....

Entry 3d7a2d567158 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 07:19:04 UTC

## Minimal Rank of Tropicalized Automorphic Representations vs. Resolution Proof Width

**Entry ID:** 3d7a2d567158 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 07:19:04 UTC

### 1. Conjecture

**Field A** (mathematical branch): Tropical Geometry (Tropicalization) × Automorphic Representation Theory **Field B** (complexity object): Resolution Proof Complexity

#### Statement:

The minimal rank of the tropicalized automorphic representations associated with a CNF instance is linearly correlated with its resolution proof width, such that  $\text{MinimalRank}(\tau_{\varphi_G}) = \Theta(w(\varphi_G))$ , where  $\tau_{\varphi_G}$  denotes the tropicalization of the automorphic representation  $\varphi_G$  derived from the CNF instance  $G$ .

#### Rationale (proposer's reasoning):

Tropicalization has been used to analyze algebraic structures in computational complexity by providing a real-algebraic geometric framework. Automorphic representations are complex algebraic objects with deep connections to number theory and L-functions, which might carry information about the structure of resolution proofs. This conjecture aims to bridge these two fields by proposing that the tropicalized automorphic representations could reflect the complexity of the resolution process.

**Taxonomy category:** TROPICAL\_AUTOMORPHIC (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** 63f03cd3d962d45c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

The conjecture is supported if, across 30 independent random seeds, the correlation coefficient between  $\text{MinimalRank}(\tau_{\varphi_G})$  and  $w(\varphi_G)$  exceeds 0.8 for at least 80% of the CNF instances  $G$  with a mean difference in  $\text{MinimalRank}(\tau_{\varphi_G})$  from its expected value being less than or equal to 3.

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict   | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|-----------|------------|------------------|------------------|
| RELATIVIZATION | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |
| NATURAL_PROOFS | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | UNCERTAIN | 0.80       | HITS             | UNCERTAIN        |
| KARP_LIPTON    | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |

### 4. Novelty audit

**Verdict:** NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

### 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

#### 5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2 * n):
            clause = [random.randint(-n, -1), random.randint(1, n)]
            if random.choice([True, False]):
                clause = [-x for x in clause]
            clauses.append(clause)
        return clauses

    def dpll(cnf):
        def solve(model):
            if not cnf:
                return model
            literal = next((l for l in range(1, n + 1) if l not in model and -l not in model), None)
            if literal is None:
                return None
            new_model = model.copy()
            new_model[literal] = True
            result = solve(new_model)
            if result is not None:
                return result
            new_model[literal] = False
            new_model[-literal] = True
            return solve(new_model)
        n = len(cnf[0])
        return solve({})
```

```

def automorphic_representation(cnf):
    # Placeholder for actual implementation of automorphic representation
    return random.randint(1, 10)

def tropicalize(rep):
    # Placeholder for actual implementation of tropicalization
    return rep

def resolution_width(cnf):
    model = dpll(cnf)
    if model is None:
        return float('inf')
    width = 0
    for clause in cnf:
        if not any(lit in model and not model[lit] for lit in clause):
            width += 1
    return width

results = []
instances_tested = 0
n_max = 0
rank_sum = 0
width_sum = 0

for n in [5, 10, 15, 20, 30, 40]:
    for _ in range(5):
        cnf = generate_cnf(n)
        instances_tested += 1
        n_max = max(n_max, n)
        rep = automorphic_representation(cnf)
        tau_rep = tropicalize(rep)
        rank = len(tau_rep) # Placeholder for actual minimal rank calculation
        width = resolution_width(cnf)
        results.append((rank, width))
        rank_sum += rank
        width_sum += width

if not results:
    return {
        "metric_name": "MinimalRank",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "No instances generated"
    }

rank_mean = Fraction(rank_sum, instances_tested)
width_mean = Fraction(width_sum, instances_tested)

correlation = sum((r - rank_mean) * (w - width_mean) for r, w in results) / (instances_tested)

return {
    "metric_name": "MinimalRank",
    "metric_value": correlation,

```

```

        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": abs(correlation) > 0.8 and all(abs(r - rank_mean) <= 3 for r, _ in res
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 7 for i in range(5, 30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(re
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results if r["metric_va
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fra
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fra
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_ho
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8ea469ac.py", line 115, in <module>
    trial_result = run_trial(seed)
                   ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8ea469ac.py", line 79, in run_trial
    rank = len(tau_rep) # Placeholder for actual minimal rank calculation
           ~~~~~
TypeError: object of type 'int' has no len()

```

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

## Reasoning:

The test crashed before producing data, which means it was unable to compute the correlation coefficient and other metrics required to evaluate the conjecture. | next: Re-run the test with proper error handling to ensure that it can complete without crashing.

## 11. Audit log (LLM calls)

**Total LLM calls:** 12

| #  | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|----|-----------------|---------------|------------------|-----------|-----|--------------|
| 1  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 15016        |
| 2  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 17775        |
| 3  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 12940        |
| 4  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 13767        |
| 5  | preregistration | ollama_remote | glm4:latest      | 0         | 0   | 10144        |
| 6  | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 8574         |
| 7  | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 8465         |
| 8  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 13042        |
| 9  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 16150        |
| 10 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 10689        |
| 11 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 19034        |
| 12 | judge           | ollama_remote | glm4:latest      | 0         | 0   | 22915        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 168511 ms total latency. Provider mix: {'ollama\_remote': 12}

*(full prompt+response transcripts available in research/audit/3d7a2d567158.jsonl)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/3d7a2d567158.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** research/pvsnp\_sandbox/test\_\*.py (per-cycle)

**Replay tarball:** research/replay/3d7a2d567158.tar.gz (if generated)